

Replication Plan

A Portfolio Approach to Massively Parallel Bayesian Optimization

OSTI ID: 2571540

Auto-generated Replication Blueprint

April 8, 2026

Contents

1	Paper Overview	2
2	Detailed Workflow	2
2.1	Phase 1: Bayesian Optimization Framework	2
2.2	Phase 2: Portfolio Allocation Strategy	2
2.3	Phase 3: Benchmark Functions	2
2.4	Phase 4: Experimental Runs	3
2.5	Phase 5: Analysis and Validation	3
3	Datasets	3
4	Models, Tools, and Software	3
4.1	Hardware Requirements	4

Paper Overview

This paper proposes a portfolio-based strategy for massively parallel Bayesian optimization (BO). Instead of relying on a single acquisition function, the method maintains a portfolio of diverse acquisition functions (e.g., Expected Improvement, Upper Confidence Bound, Thompson Sampling, Knowledge Gradient) and allocates the parallel batch budget across them using a multi-armed bandit or portfolio optimization framework (e.g., based on the Hypervolume Sharpe Ratio). This approach provides robustness against the “exploration–exploitation” dilemma and enables efficient use of hundreds to thousands of parallel evaluations.

Detailed Workflow

Phase 1: Bayesian Optimization Framework

1.1 Implement or use a GP-based Bayesian optimization library.

- Gaussian process surrogate model with specified kernel (e.g., Matérn 5/2).
- Hyperparameter optimization via marginal likelihood.

1.2 Implement individual acquisition functions.

- Expected Improvement (EI).
- Upper Confidence Bound (UCB) with varying β .
- Thompson Sampling (TS).
- Knowledge Gradient (KG).
- Probability of Improvement (PI).

Phase 2: Portfolio Allocation Strategy

2.1 Implement the portfolio allocation mechanism.

- Multi-armed bandit approach (e.g., Exp3, Hedge).
- Hypervolume Sharpe Ratio (HVSr) for multi-objective allocation.
- Uniform allocation baseline.

2.2 Batch generation.

- For a batch of B parallel evaluations, allocate B_i to acquisition function i .
- Generate candidate points by optimizing each acquisition function.
- Deduplicate and ensure diversity in the batch.

Phase 3: Benchmark Functions

3.1 Implement standard benchmark functions.

- Branin, Hartmann-6D, Ackley, Rosenbrock, Levy, Styblinski-Tang.
- Higher-dimensional test functions as specified.

3.2 Implement real-world surrogate benchmarks (if used in the paper).

Phase 4: Experimental Runs

- 4.1 **Run the portfolio BO** on each benchmark for multiple random seeds.
- 4.2 **Run baselines:** single-acquisition-function BO (EI-only, UCB-only, TS-only), random search.
- 4.3 **Vary batch sizes** (e.g., $B = 8, 16, 32, 64, 128, 256$).
- 4.4 **Record:** best observed value vs. iteration, wall-clock time, portfolio weights over time.

Phase 5: Analysis and Validation

- 5.1 **Compute performance metrics:** simple regret, cumulative regret, normalized improvement.
- 5.2 **Statistical comparison:** mean and confidence intervals over random seeds.
- 5.3 **Reproduce all figures and tables** from the paper.
- 5.4 **Analyze portfolio weight dynamics.**

Datasets

Dataset / Source	Type	Location / Notes
Benchmark test functions	Synthetic	Standard definitions; implemented programmatically
Real-world surrogates (if used)	Public	HPOBench, Bayesmark, or as specified https://github.com/automl/HPOBench

Models, Tools, and Software

Tool / Library	Version	Purpose / URL
BoTorch	≥ 0.8	Bayesian optimization in PyTorch https://botorch.org/
GPyTorch	≥ 1.9	Gaussian process modeling https://gpytorch.ai/
PyTorch	≥ 1.12	Deep learning backend
Ax Platform	latest	High-level BO framework (uses BoTorch) https://ax.dev/
NumPy / SciPy	≥ 1.21	Numerical operations, optimization
Matplotlib / Seaborn	≥ 3.4	Performance comparison plots

Tool / Library	Version	Purpose / URL
scikit-optimize	latest	Alternative BO library https://scikit-optimize.github.io/

Hardware Requirements

- Multi-core workstation for parallel batch evaluation.
- GPU optional (useful for GP fitting with GPyTorch on large datasets).
- Each benchmark run: minutes to hours depending on batch size and function evaluation cost.